

Applications of On-chip Trace on Debugging Embedded Processor

Xiao Hu

Shuming Chen

*School of Computer, National University of Defense Technology,
Changsha, Hunan, P. R. of China, 410073
xiaohu@nudt.edu.cn*

Abstract

On-chip trace system records run-time information of the embedded processor with special hardware to overcome the obstacle of non-intrusive debug and optimization of traditional techniques. This paper explores the mechanism, characteristics and applications of on-chip trace technique with TraceDo, an on-chip trace system of a multi-core SoC. The functions and structures of TraceDo are introduced, the working process of path trace is explained and the trace applications with two cases of debug and optimization are also discussed.

1. Introduction

The increasing demand for embedded systems makes embedded software more complex [26]. Developers are increasingly overwhelmed by development challenges such that system reliability is decreasing. However, more complex software is not the only challenge to developers. The traditional debug tools are not efficient enough to keep up with new characteristics of embedded systems under time-to-market pressures.

The software simulator is a debug tool with low cost. But the complex interfaces in embedded systems and variant types of continual data communication through these interfaces make modeling real systems imprecisely.

Breakpoints and single stepping (Bp&ss) are fundamental debug methods. But they change software behavior in the real-time systems [35], and the concurrency in multi-processor systems is hard to be watched using traditional debug techniques [34]. In control systems, mechanical parts are easily damaged or out of control by stopping systems suddenly using breakpoints.

Logic analyzers do not change software behavior. But on higher integration of SoCs and processors with

on-chip cache, board-level interfaces which could previously be monitored and debugged with logic analyzers are now buried in silicon [27] [29].

Software instruments and profiling are popular technique for software debug and optimizations. Information on program execution is collected by adding instructions to source code [18]. This method is intrusive and consumes excessive system resources such as CPU cycles and memories [33]. This limits its utilizations in resource-sensitive embedded systems. Software behavior in real-time or parallel systems may be also changed by the instructions added.

When these problems have to be solved by additional area and power, on-chip trace technique has become popular in recent years. The leading processor-core vendors such as ARC, ARM and MIPS Technologies provide their solutions of on-chip trace [1] [2] [3] [4]. The IEEE-ISTO NEXUS 5001 STD also includes protocols to support on-chip trace [5]. A survey and detailed comparisons of leading on-chip trace techniques are given in [21] [22]. The on-chip trace system non-intrusively records real-time information such as program path and data accesses by special hardware. After being compressed to trace data streams, this information is stored on the chip or transferred to the Debug Host PC through special data path, I/O port and the out-chip emulator. The software tools in the host PC decompress trace data and recover run-time information for debug and Optimization.

The relationships of above five debug techniques are shown in Figure 1.

The key issues of on-chip trace include: what information should be recorded, how to compress, transfer and store trace data, and how to apply trace information. Though architectures and performances of on-chip trace are researched by [20] [22] in recent years, but its intrinsic advantages, mechanisms and applications are seldom discussed before. This paper compares five state-of-the-art debug techniques and analyzes the leverage of complementary advantages of trace and Bp&ss. The functions and structures of on-

chip trace are introduced by a concrete trace system (TraceDo). A detailed working mechanism of program path trace is explained. The application areas and two cases of debug and optimization are also discussed.

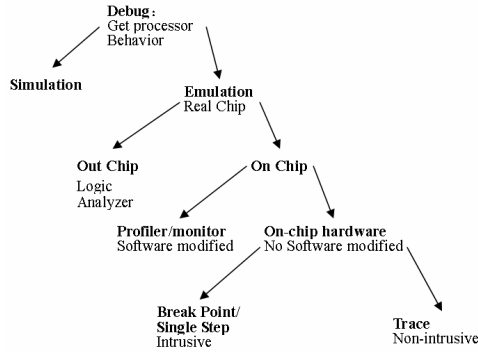


Figure 1. Relationships of debug techniques.

2. TraceDo system

The YHFT-DSP is a series of high performance VLIW (Very Long Instruction Word) DSPs (Digital Signal Processor) [12]. The development tools of YHFT-DSP include on-chip debug hardware, Emulators with I/O ports of the enhanced JTAG or the host processor interface, software simulators, and the Integrated Development Environment (IDE) with graphic interfaces [39][11]. YHFT-QDSP is a heterogeneous multiprocessor SoC integrated a commercial RISC core and four DSP cores of YHFT-DSP/700 [12]. For overcoming the shortage of Bp&ss and giving the real-time visibility of DSP cores for debug and optimization, an on-chip trace system named TraceDo (Trace for Debug and Optimization) is designed for YHFT-QDSP, as shown in Figure 2. TraceDo is modular and scalable, and it is able to be used in all YHFT DSPs.

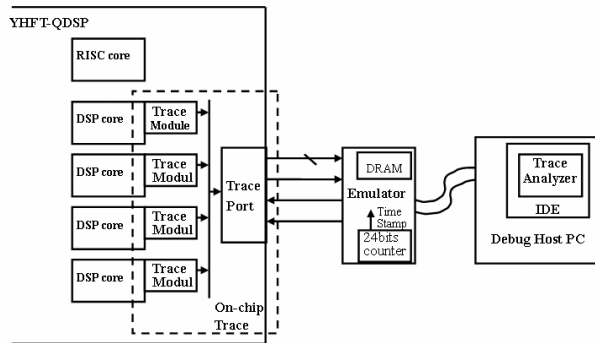


Figure 2. Structure of TraceDo.

2.1. Functions and advantages

TraceDo comprises the on-chip trace hardware, the Emulator and the software tool called Trace Analyzer embedded in the YHFT-DSP IDE. Trace Module of each DSP core records program path (Path Trace), data access (Data Trace) and peripheral events (Event Trace), and compresses these informations into trace messages with appropriate compression schemes respectively. The trace messages are transferred to Trace Port through Trace Bus. The Emulator receives trace messages on Trace Port and attaches out-chip timestamps to them. Precise time marks of messages are recovered by out-chip timestamps and on-chip delay stamps contained in trace messages. Trace messages are stored in DRAM buffers of Emulator or in the Debug Host PC. The Trace Analyzer decompresses and analyzes trace messages to recover the program behavior for debug and optimization.

The message format of TraceDo is various lengths aligned on byte boundaries, as shown in Figure 3. Such a message format effectively reduces overhead bits for headers and for indicating message length of short messages with high probability. TraceDo compresses values of address and data in the XOR scheme by avoiding transferring the redundancy high bytes that are the same to the last transferred value. A LS Encoder is proposed for compressing results of conditional direct branches by bit mapping and length encoding efficiently. The novel configurations of Enable Bits, Force Bits and Degrade Bits are proposed for configuring branch messages flexibly. These approaches much improve the compression ratios and flexibilities. Besides popular trace contents of program path and data access, TraceDo also records peripheral events such as pipeline stalls and DMA (Direct Memory Access) as Event Trace for optimization. With a separated encoding scheme for pipeline stall messages, users can achieve a good trade-off between enough precision and bandwidth consumed. All configuration registers of on-chip trace hardware can be accessed by JTAG instructions and program instructions of Load and Store. A novel NOP_config instruction is also designed to configure TraceDo non-intrusively. A detailed structure and performance analysis of TraceDo can be found in [22].

1 st Byte		2 nd Byte (option)		...	Last Byte (option)
Header	F bit	Data Sect	F bit	Data Sect	Data Sect

Header: message type

F(Follow bit)=1: no following Byte

Data Sect: message data

Figure 3. Message structure of TraceDo.

Gupta [18] give good summaries of profile applications.

The main applications of on-chip trace are listed:

- Path-based optimization. The program execution path can be exploited for performing path sensitive optimizations [37] [8] [17] and path-sensitive prediction techniques [24].
- Instruction memory optimization. For processors with instruction cache, the program execution path is important for reducing cache conflict miss by code layout [16].
- Debug. Software development can be accelerated by making debuggers more path sensitive and give programmers a path-based query facility for directing the debugging process [9]. Comparing spectra from different executions of a program can identify places in which the program's behavior varies with its input for isolating bugs [31].
- Testing. Path trace can be used in automated test-case generation [15] and improve path-based test coverage [14].
- Software complexity criteria and program understanding [30].
- Data trace. The value of data is used to perform code specialization [10], data compression [40], value speculation [28], and value encoding [36]. The address of data access can be used in identifying hot data streams that exhibit data locality which can help in finding cache conscious data layouts [32] and developing data prefetching mechanisms [25] [13]. Data trace is also useful for fault diagnoses.

4.1. Debug using TraceDo

The important use of TraceDo is hard to quantify, namely how it helps software developers fix production bugs. So in this paper, several example uses of TraceDo for fault diagnoses are offered in the migration of an image recognitions program to YHFT-QDSP.

- The program compiled often crashed initially. When using Bp&ss, stepping in lines and in functions or setting multiple breakpoints are used for the program execution path and understanding programs. When using TraceDo, running the program only one time give the whole hierarchy execution path and the execution time of each function. Compared with the correct original execution path, the bug is narrowed down and is fixed by Bp&ss easily. The problem is the wrong configurations of the memory interface caused the overlap of memory addresses.

- When multicores work together, the breakpoints terminate the program running, cause different interrupt behaviors and tasks scheduling and cause synchronization problems with other cores. By using TraceDo, run-time execution informations of multicores are acquired without terminating the system running.
- The original program is instrumented with more than seventy instrument instructions inserted for recording real-time execution behaviors. The consumptions of the memory and CPU cycles caused by these instructions are much avoided by using TraceDo.

4.2. Optimization using TraceDo

An application example of the JPEG encoder running in one DSP core is used for optimization using TraceDo. The path trace, event trace of PStall (pipeline stall for fetching instruction) and DStall (pipeline stall for fetching data) and out-chip timestamps are enabled before the program and TraceDo run. After encoding two graphs, the program goes to a breakpoint and the trace file is dumped from the DRAM in Emulator. Run-time behaviors of the program are revealed by analyzing the trace file. A section of the program execution path at function level and the execution time ratios of functions with PStall/DStall are shown in Figure 5 and Figure 6 separately. The execution orders, execution counts and execution time of each function can also be revealed, and exceptions of program behavior are easy to be found.

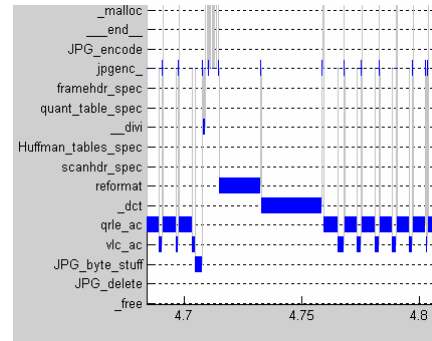


Figure 5. A section of program execution path. The function names are listed in y-axis. The x-axis is the time axis.

As shown in Figure 6, the three functions with most execution time (*dct*, *qrle_ac*, *vlc_ac*) have been optimized in assemble language, so these codes can be hardly improved further. When analyzing the functions of *memcpy* and *reformat* with highest ratio of DStall, the instructions of copying data in *memcpy* are

replaced by DMA operations for efficiency, and data layout are implemented in *reformat* for reducing data cache conflict miss. When analyzing the function of *jpgenc* with highest ratio of PStall, many conditional branches and function calls written in C language are optimized by code layout for avoiding instruction cache conflicts. With above approaches of optimization, the total execution time of the JPEG application is reduced by 7.2%.

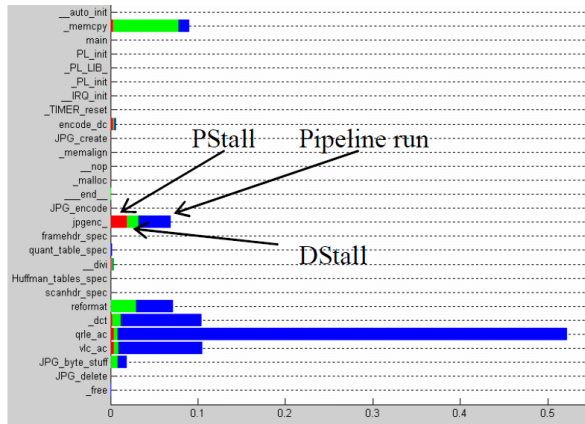


Figure 6. Time ratios of function execution and PStall/DStall. The function names listed in y-axis. The ratios of execution time are labeled in x-axis.

5. Conclusion

The non-intrusive debug of embedded software is a hard problem for traditional debug techniques. The on-chip trace is a useful technique to conquer it by special hardware, while the infrastructures and applications of on-chip trace are seldom studied before. On-chip trace is a non-intrusive debug technique without software modification. It is suitable for debug and optimization with timing constrain and concurrency and it has the complementary advantage to Bp&ss another debug technique using on-chip hardware. On-chip trace has many applications such as debug, testing, codes optimization and program understanding etc. Functions and structures of a concrete on-chip trace system TraceDo is introduced in this paper. Path trace working process and two cases of trace applications are also discussed.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (60473079), SRFDP (No.20059998026).

References

- [1] ARC International Ltd. ARC International Provides New Configurable Trace and Debug Extensions to the ARCTM 600 and 700 Core Families. <http://www.arc.com/news/PressRelease.html?id=227>, 2005-11-29
- [2] ARM Ltd. CoreSight Flyer. <http://www.arm.com/products/solutions/CoreSight.html>, 2004-10-1
- [3] MIPS Technologies Inc. The PDtraceTM Interface and Trace Control Block Specification. <http://www.mips.com/content/Documentation/MIPSDocumentation/ProcessorArchitecture/doclibrary#ArchitectureSetExtensions>, 2005-7-4
- [4] Freescale Ltd. MPC565 Reference Manual. http://www.freescale.com/webapp/sps/site/prod_summary.jsp?code=MPC565, 2005-11
- [5] IEEE-ISTO 5001TM-2003, The Nexus 5001 ForumTM Standard for a Global Embedded Processor Debug Interface v2.0. <http://www.nexus5001.org/standard2.html>, 2003-11
- [6] J. Anderson, et al., Continuous profiling: Where have all the cycles gone?, Proceedings of the 16th ACM Symposium of Operating Systems Principles, October 1997
- [7] T. Ball, and J. Larus, Using paths to measure, explain, and enhance program behavior, Computer, July, 2000
- [8] R. Bodik, R. Gupta and M.L. Soffa, Complete Removal of Redundant Expressions, ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pages 1-14, Montreal, Canada, June 1998
- [9] B. Bruegge, P. Hibbard, Generalized Path Expressions: A High-level Debugging Mechanism, J. Systems and Software, Vol. 3, No. 4, 1983
- [10] B. Calder, P. Feller and A. Eustace, Value Profiling, The 30th IEEE/ACM International Symposium on Microarchitecture (MICRO), pages 259-269, December 1997
- [11] Lili Chen, The Design and Realization of Development Environments for YHFT-DSP, M.S. dissertation, National University of Defense Technology, Changsha, China, Dec, 2006
- [12] Shuming Chen, Zhentao Li, Jianghua Wan, Dinglei Hu, Yang Guo, Dong Wang, Xiao Hu and Shuwei Sun. Research and Development of High Performance YHFT Digital Signal Processor, Journal of Computer Research and Development, 2006, 43(6)
- [13] T.M. Chilimbi and M. Hirzel, Dynamic Hot Data Stream Prefetching for General-Purpose Programs, ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2002

- [14] L.A. Clarke et al., A Formal Evaluation of Data Flow Path Selection Criteria, *IEEE Trans. Software Eng.*, Vol. 15, No. 11, 1989
- [15] L.A. Clarke, A System to Generate Test Data and Symbolically Execute Programs, *IEEE Trans. Software Eng.*, Vol. 2, No. 3, 1976
- [16] Nikolas Gloy, Michael D. Smith, Procedure Placement Using Temporal-Ordering Information, *ACM Transactions on Programming Languages and Systems*, Vol. 21, No. 5, September 1999
- [17] R. Gupta, D. Berson, and J.Z. Fang, Path Profile Guided Partial Redundancy Elimination Using Speculation, *IEEE International Conference on Computer Languages (ICCL)*, pages 230-239, Chicago, Illinois, May 1998
- [18] Rajiv Gupta, Eduard Mehofer, and Youtao Zhang. Profile guided compiler optimizations. In *The Compiler Design Handbook: Optimizations and Machine Code Generation*. CRC Press, 2002
- [19] S. Hangal and M. S. Lam, Tracking down software bugs using automatic anomaly detection. In *International Conference on Software Engineering*, May 2002
- [20] Hopkins, ABT, McDonald-Maier, KD. Debug Support Strategy for Systems-on-Chips with Multiple Processor Cores, *IEEE Trans. on Computers*, 2006, 55(2)
- [21] Xiao Hu, Shuming Chen. A Survey to On-chip Trace Systems for Real-time Debug in Embedded Processors, *NCCET'06*, Guilin, 8, 2006
- [22] Xiao Hu, et al., TraceDo: An On-Chip Trace System for Real-Time Debug and Optimization in Multiprocessor SoC, *ISPA'06*, Italy. Dec, 2006
- [23] Joel Huselius, Debugging parallel systems: A state of the art report, Technical Report 63, Mlardalen University, Department of Computer Science and Engineering, September 2002
- [24] Q. Jacobson, E. Rotenberg, and J.E. Smith, Path-Based Next Trace Prediction, *The 30th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, December 1997
- [25] D. Joseph and D. Grunwald, Prefetching Using Markov Predictors, *International Symposium on Computer Architecture (ISCA)*, pages 252–263, 1997
- [26] Edward A. Lee.: What's Ahead for Embedded Software, *IEEE Computer*. 9, 2000
- [27] R. Leatherman and N. Stollon, An embedded debugging architecture for SoCs, *IEEE Potentials*, Feb/Mar 2005
- [28] M. H. Lipasti and J. P. Shen, Exceeding the Dataflow Limit via Value Prediction, *29th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 226-237, Dec. 1996
- [29] Robert F. Molyneaux, Debug and Diagnosis in the Age of System-on-a-Chip, *ITC*, 2003
- [30] B.A. Nejme, NPATH: A Measure of Execution Path Complexity and Its Applications, *Comm. ACM*, Feb. 1988
- [31] T. Reps et al., The Use of Program Profiling for Software Maintenance with Applications to the Year 2000 Problem, *Proc. Fifth ACM SIGSOFT Symp. Foundations of Software Eng.*, 1997
- [32] S. Rubin, R. Bodik, and T. Chilimbi, An Efficient Profile Analysis Framework for Data Layout Optimizations, *The 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, Portland, Oregon, Jan. 2002
- [33] S. Savage, et al., Eraser: A dynamic data race detector for multithreaded programs, *ACM Transactions on Computer Systems*, November 1997
- [34] Darlene Stewart and Morven Gentleman, Non-stop monitoring and debugging on shared memory multiprocessors, In *Proceedings of the 2nd International Workshop on Software Engineering for Parallel and Distributed Systems*, *IEEE Computer Society*, May 1997
- [35] Daniel Sundmark, Deterministic Replay Debugging of Embedded Real-Time Systems using Standard Components, PhD. Thesis, Department of Computer Science and Engineering Mälardalen University Vasteras, Sweden, March 2004
- [36] J. Yang, R. Gupta, Frequent Value Locality and its Applications, *ACM Transactions on Embedded Computing Systems (TECS)*, special inaugural issue on memory systems, Vol. 1, No. 1, pages 79-105, November 2002
- [37] C. Young and M.D. Smith, Better Global Scheduling Using Path Profiles, *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 115-123, 1998
- [38] Xiangyu Zhang, Rajiv Gupta, Whole Execution Traces, *37th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2004
- [39] Xinfang Zhang, The Design and Realization of On-Chip Debug Structure for YHFT-DSP, M.S. dissertation, National University of Defense Technology, Changsha, China, June, 2006
- [40] Y. Zhang and R. Gupta, Data Compression Transformations for Dynamically Allocated Data Structures. *International Conference on Compiler Construction (CC)*, Grenoble, France, April 2002